

**LAPORAN PRAKTIKUM
(TUGAS 1 SISTEM OPERASI)**



**Dosen Pengampu :
Triawan Adi Cahyanto M.kom**

**Disusun oleh:
MUHAMMAD RIDHO (2410651009) (A)**

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025**

1. Tujuan Praktikum

Untuk memahami konsep manajemen proses dalam sistem operasi, termasuk:

- Pembuatan proses menggunakan `fork()`
- Sinkronisasi antar proses menggunakan `waitpid()`
- Komunikasi antar proses (IPC) menggunakan pipe dan shared memory
- Pengukuran waktu eksekusi proses menggunakan `gettimeofday()`
- Penanganan status exit dari proses anak

2. Dasar Teori

2.1. Fork()

`fork()` digunakan untuk membuat proses baru (child process) dari proses induk (parent).

Setelah `fork()`, dua proses berjalan paralel dengan ruang memori terpisah.

2.2. Waitpid()

`waitpid()` digunakan oleh proses induk untuk menunggu proses anak selesai dieksekusi.

Dengan ini, zombie process dapat dihindari karena status exit anak dikumpulkan oleh parent.

2.3. Pipe()

`pipe()` merupakan mekanisme IPC unidirectional, digunakan agar parent dapat menerima data dari child.

2.4. Shared Memory

`shmget()` dan `shmat()` digunakan untuk membuat dan mengakses memori bersama antar proses.

Shared memory memungkinkan komunikasi lebih cepat karena data disimpan di memori, bukan dikirim lewat file descriptor.

2.5. Zombie Process

Zombie process adalah proses anak yang telah selesai dieksekusi tetapi belum diambil statusnya oleh parent.

Zombie process dihindari dengan `wait()` atau `waitpid()`.

3. Deskripsi Program

Program ini merupakan **manajer proses sederhana** dengan tiga child process:

1. **Child 1:** Menghitung faktorial dari angka input user.
2. **Child 2:** Menghitung bilangan prima hingga batas tertentu.
3. **Child 3:** Membalik string yang dimasukkan user.

Parent process akan:

- Menunggu semua child selesai (waitpid())
- Mengukur waktu eksekusi (gettimeofday())
- Menampilkan hasil dari setiap child melalui **pipe** (child 1 dan 2)
- Menampilkan hasil dari **shared memory** (child 3, karena NIM ganjil)

4. Flowchart Program

1. Mulai
2. Input 3 data dari user
3. Buat 3 pipe dan 1 shared memory
4. Fork 3 child
5. Tiap child mengerjakan tugas masing-masing
6. Parent menunggu semua child (waitpid)
7. Parent membaca hasil dari pipe & shared memory
8. Tampilkan waktu eksekusi
9. Selesai

- CODE SOURCE

```
root@RIDHO: ~
GNU nano 4.8 progress_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define MAX_BUF 4096

double elapsed_sec(struct timeval start, struct timeval end) {
    double s = (double)(end.tv_sec - start.tv_sec);
    double us = (double)(end.tv_usec - start.tv_usec);
    return s + us/1e6;
}

unsigned long long factorial(unsigned int n) {
    unsigned long long res = 1ULL;
    for (unsigned int i = 2; i <= n; ++i) res *= i;
    return res;
}

int is_prime(int x) {
    if (x < 2) return 0;
    if (x % 2 == 0) return (x == 2);
    for (int i = 3; i * i <= x; i += 2) if (x % i == 0) return 0;
    return 1;
}

char* primes_upto(int limit) {
    char *buf = malloc(MAX_BUF);
    buf[0] = '\0';
    size_t off = 0;
    for (int i = 2; i <= limit; ++i) {
        if (is_prime(i)) {
            int n = snprintf(buf+off, MAX_BUF-off, "%d ", i);
            off += n;
            if (off > MAX_BUF - 50) break;
        }
    }
    return buf;
}

int main(void) {
    unsigned int n_fact;
    int primes_limit;
    char input_str[100];

    printf("=== Process Manager (NIM 2410651009 - Ganjil: Shared Memory) ===\n");
    printf("Masukkan angka untuk faktorial (Child 1): ");
    scanf("%u", &n_fact);
    printf("Masukkan batas bilangan prima (Child 2): ");
    scanf("%d", &primes_limit);
    printf("Masukkan string untuk dibalik (Child 3): ");
    scanf("%s", input_str);

    printf("\nInput Child 1: %u\n", n_fact);
    printf("Input Child 2: %d\n", primes_limit);
    printf("Input Child 3: %s\n", input_str);

    int pfd[3][2];
    for (int i = 0; i < 3; ++i) if (pipe(pfd[i]) == -1) return 1;

    int shmid = shmget(IPC_PRIVATE, MAX_BUF, IPC_CREAT | 0600);
    if (shmid == -1) return 1;
    char *shm_parent = shmat(shmid, NULL, 0);
```

```
root@RIDHO: ~
GNU nano 4.8 progress_manager.c
char* primes_upto(int limit) {
    char *buf = malloc(MAX_BUF);
    buf[0] = '\0';
    size_t off = 0;
    for (int i = 2; i <= limit; ++i) {
        if (is_prime(i)) {
            int n = snprintf(buf+off, MAX_BUF-off, "%d ", i);
            off += n;
            if (off > MAX_BUF - 50) break;
        }
    }
    return buf;
}

int main(void) {
    unsigned int n_fact;
    int primes_limit;
    char input_str[100];

    printf("=== Process Manager (NIM 2410651009 - Ganjil: Shared Memory) ===\n");
    printf("Masukkan angka untuk faktorial (Child 1): ");
    scanf("%u", &n_fact);
    printf("Masukkan batas bilangan prima (Child 2): ");
    scanf("%d", &primes_limit);
    printf("Masukkan string untuk dibalik (Child 3): ");
    scanf("%s", input_str);

    printf("\nInput Child 1: %u\n", n_fact);
    printf("Input Child 2: %d\n", primes_limit);
    printf("Input Child 3: %s\n", input_str);

    int pfd[3][2];
    for (int i = 0; i < 3; ++i) if (pipe(pfd[i]) == -1) return 1;

    int shmid = shmget(IPC_PRIVATE, MAX_BUF, IPC_CREAT | 0600);
    if (shmid == -1) return 1;
    char *shm_parent = shmat(shmid, NULL, 0);
```

```
root@RIDHO: ~
GNU nano 4.8 progress_manager.c
int shm_id = shmget(IPC_PRIVATE, MAX_BUF, IPC_CREAT | 0600);
if (shm_id == -1) return 1;
char *shm_parent = shmat(shm_id, NULL, 0);
shm_parent[0] = '\0';

pid_t pids[3];
struct timeval start[3], end[3];

for (int i = 0; i < 3; ++i) {
    pid_t pid = fork();
    if (pid == 0) {
        for (int j = 0; j < 3; ++j) {
            if (j != i) {
                close(pfd[j][0]);
                close(pfd[j][1]);
            } else {
                close(pfd[j][0]);
            }
        }

        if (i == 0) {
            unsigned long long res = factorial(n_fact);
            char msg[MAX_BUF];
            snprintf(msg, sizeof(msg), "Child1 (input %u): hasil faktorial = %llu\n", n_fact, res);
            write(pfd[i][1], msg, strlen(msg));
            close(pfd[i][1]);
            _exit(0);
        }

        else if (i == 1) {
            char *primes = primes_upto(primes_limit);
            char msg[MAX_BUF];
            snprintf(msg, sizeof(msg), "Child2 (input %d): bilangan prima = %s\n", primes_limit, primes);
            write(pfd[i][1], msg, strlen(msg));
            free(primes);
            close(pfd[i][1]);
            _exit(0);
        }
    }
}
```

Get Help Exit Write Out Read File Where Is Replace Cut Text Paste Text Justify To Spell Cur Pos Go To Line Undo Redo Mark Text Copy Text To Bracket Where Was

```
root@RIDHO: ~
GNU nano 4.8 progress_manager.c
    else {
        char reversed[100];
        int len = strlen(input_str);
        for (int k = 0; k < len; k++)
            reversed[k] = input_str[len - 1 - k];
        reversed[len] = '\0';
        char *shm_child = shmat(shm_id, NULL, 0);
        snprintf(shm_child, MAX_BUF, "Child3 (input %s): hasil dibalik = %s\n", input_str, reversed);
        shmdt(shm_child);
        close(pfd[i][1]);
        _exit(0);
    }
} else {
    close(pfd[i][1]);
    gettimeofday(&start[i], NULL);
    pids[i] = pid;
}

for (int i = 0; i < 3; ++i) {
    int status;
    waitpid(pids[i], &status, 0);
    gettimeofday(&end[i], NULL);
    double t = elapsed_sec(start[i], end[i]);
    printf("\n[Parent] Child %d (PID %d) selesai dalam %.6f detik\n", i+1, pids[i], t);
    if (i == 2) {
        printf("Hasil (shared memory):\n%s", shm_parent);
    } else {
        char buf[512];
        ssize_t n;
        printf("Hasil (pipe):\n");
        while ((n = read(pfd[i][0], buf, sizeof(buf)-1)) > 0) {
            buf[n] = '\0';
            printf("%s", buf);
        }
        close(pfd[i][0]);
    }
}
```

Get Help Exit Write Out Read File Where Is Replace Cut Text Paste Text Justify To Spell Cur Pos Go To Line Undo Redo Mark Text Copy Text To Bracket Where Was

```
root@RIDHO: ~
GNU nano 4.8 progress_manager.c

    shmdt(shm_child);
    close(pfd[i][1]);
    _exit(0);
}
} else {
    close(pfd[i][1]);
    gettimeofday(&start[i], NULL);
    pids[i] = pid;
}
}

for (int i = 0; i < 3; ++i) {
    int status;
    waitpid(pids[i], &status, 0);
    gettimeofday(&end[i], NULL);
    double t = elapsed_sec(start[i], end[i]);
    printf("\n[Parent] Child %d (PID %d) selesai dalam %.6f detik\n", i+1, pids[i], t);
    if (i == 2) {
        printf("Hasil (shared memory):\n%s", shm_parent);
    } else {
        char buf[512];
        ssize_t n;
        printf("Hasil (pipe):\n");
        while ((n = read(pfd[i][0], buf, sizeof(buf)-1)) > 0) {
            buf[n] = '\0';
            printf("%s", buf);
        }
        close(pfd[i][0]);
    }
}

shmdt(shm_parent);
shmctl(shmid, IPC_RMID, NULL);
printf("\n=== Semua proses selesai ===\n");
return 0;
}

^G Get Help  ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C Cur Pos   ^M-U Undo     ^M-A Mark Text ^M-] To Bracket
^X Exit      ^R Read File  ^N Replace    ^U Paste Text ^T To Spell   ^_ Go To Line ^M-E Redo     ^M-C Copy Text ^M-_ Where Was
```

- OUTPUT YANG DIHASILKAN

```
root@RIDHO: ~
Input Child 3: Ridho

[Parent] Child 1 (PID 5826) selesai dalam 0.000702 detik
Hasil (pipe):
Child1 (input 5): hasil faktorial = 120

[Parent] Child 2 (PID 5827) selesai dalam 0.000537 detik
Hasil (pipe):
Child2 (input 20): bilangan prima = 2 3 5 7 11 13 17 19

[Parent] Child 3 (PID 5828) selesai dalam 0.000508 detik
Hasil (shared memory):
Child3 (input Ridho): hasil dibalik = ohdiR

=== Semua proses selesai ===
root@RIDHO:~# nano progress_manager.c
root@RIDHO:~# gcc -o progress_manager progress_manager.c
root@RIDHO:~# ./progress_manager
=== Process Manager (MM 2010651009 - Ganjil: Shared Memory) ===
Masukkan angka untuk faktorial (Child 1): 5
Masukkan batas bilangan prima (Child 2): 20
Masukkan string untuk dibalik (Child 3): Ridho

Input Child 1: 5
Input Child 2: 20
Input Child 3: Ridho

[Parent] Child 1 (PID 5836) selesai dalam 0.000209 detik
Hasil (pipe):
Child1 (input 5): hasil faktorial = 120

[Parent] Child 2 (PID 5837) selesai dalam 0.000201 detik
Hasil (pipe):
Child2 (input 20): bilangan prima = 2 3 5 7 11 13 17 19

[Parent] Child 3 (PID 5838) selesai dalam 0.000203 detik
Hasil (shared memory):
Child3 (input Ridho): hasil dibalik = ohdiR

=== Semua proses selesai ===
root@RIDHO:~# |
```

KESIMPULAN

`fork()` digunakan untuk membuat proses baru lalu `waitpid()` mencegah zombie process dengan menunggu proses anak. Setelah itu `pipe()` dan shared memory merupakan contoh mekanisme IPC (inter process communication). Dan penggunaan `gettimeofday()` efektif untuk mengukur durasi eksekusi tiap proses. Dan yang terakhir yaitu shared memory memberikan komunikasi yang lebih efisien daripada `pipe`, namun lebih kompleks dalam sinkronisasi.

**LAPORAN PRAKTIKUM
(TUGAS 2 SISTEM OPERASI)**



**Dosen Pengampu:
Triawan Adi Cahyanto M.kom**

**Disusun oleh:
MUHAMMAD RIDHO (2410651009)**

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
20225**

1. Tujuan Praktikum

Mengetahui cara kerja system call inotify untuk memantau aktivitas file (pembuatan, perubahan, dan penghapusan) pada direktori tertentu secara real-time, serta mencatat hasilnya dalam log file menggunakan fungsi dasar sistem (open, write, time, localtime).

2. Dasar Teori

- **System Call:**

Adalah antarmuka antara program pengguna dan kernel untuk melakukan operasi sistem (seperti manajemen file, proses, dan memori).

- **inotify (Linux):**

Sebuah mekanisme kernel Linux untuk memantau perubahan file atau direktori. Fungsi penting:

- a. `inotify_init()` : Membuat instance pemantauan.
- b. `inotify_add_watch()` : Menentukan direktori atau file yang akan diawasi.
- c. `read()` : Membaca event perubahan dari descriptor.
- d. `inotify_event` : Struktur data yang menyimpan detail event.

- **Jenis Event:**

- e. `IN_CREATE` : File/direktori baru dibuat.
- f. `IN_MODIFY` : File diubah.
- g. `IN_DELETE` : File dihapus.

- **Signal Handling (SIGTERM):**

Digunakan untuk menghentikan program secara aman dan menutup file descriptor `inotify`.

- **Shared Memory (opsional variasi tugas ganjil):**

Tidak digunakan di sini, tetapi NIM ganjil memiliki variasi berupa notifikasi dummy (email simulasi).

- INI ADALAH KODE SOURCE NYA

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <sys/inotify.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>
#include <string.h>
#include <errno.h>

static volatile sig_atomic_t tetap_jalan = 1;
static int inotify_fd = -1;

void tangani_sigterm(int sig) {
    (void)sig;
    tetap_jalan = 0;
}

ssize_t tulis_penuh(int fd, const void *buf, size_t count) {
    const char *p = buf;
    size_t sisa = count;
    while (sisa > 0) {
        ssize_t w = write(fd, p, sisa);
        if (w < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        sisa -= (size_t)w;
        p += w;
    }
    return (ssize_t)count;
}
```

```

const char *tipe_event(uint32_t mask) {
    if (mask & IN_CREATE) return "BUAT";
    if (mask & IN_MODIFY) return "UBAH";
    if (mask & IN_DELETE) return "HAPUS";
    return "LAINNYA";
}

int tambah_log(const char *path_log, const char *baris) {
    int fd = open(path_log, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd < 0) return -1;
    size_t len = strlen(baris);
    if (tuliskan(fd, baris, len) != (ssize_t)len) {
        close(fd);
        return -1;
    }
    close(fd);
    return 0;
}

void simulasi_email(const char *kepada, const char *subjek, const char *isi) {
    dprintf(STDOUT_FILENO, "--- SIMULASI EMAIL DUMMY ---\n");
    dprintf(STDOUT_FILENO, "Kepada: %s\n", kepada);
    dprintf(STDOUT_FILENO, "Subjek: %s\n", subjek);
    dprintf(STDOUT_FILENO, "Isi Pesan:\n%s\n", isi);
    dprintf(STDOUT_FILENO, "--- AKHIR SIMULASI EMAIL ---\n");
}

```

```

int main(int argc, char *argv[]) {
    const char *direktori_awas = "/tmp/monitor_dir";
    const char *path_log = "./file_monitor.log";

    if (argc >= 2) direktori_awas = argv[1];
    if (argc >= 3) path_log = argv[2];

    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = tangani_sigterm;
    sigaction(SIGTERM, &sa, NULL);

    struct stat st;
    if (stat(direktori_awas, &st) < 0) {
        if (mkdir(direktori_awas, 0755) < 0) {
            perror("mkdir direktori_awas");
            return 1;
        }
    }

    inotify_fd = inotify_init1(IN_NONBLOCK);
    if (inotify_fd < 0) {
        perror("inotify_init1");
        return 1;
    }

    int wd = inotify_add_watch(inotify_fd, direktori_awas, IN_CREATE | IN_MODIFY | IN_DELETE);
    if (wd < 0) {
        perror("inotify_add_watch");
        close(inotify_fd);
        return 1;
    }

    char buf[4096] __attribute__((aligned(8)));
}

```

```

dprintf(STDOUT_FILENO, "Memantau direktori: %s\nMencatat ke: %s\n", direktori_awas, path_log);
dprintf(STDOUT_FILENO, "Hentikan dengan SIGTERM (kill -TERM <pid>)\n");

while (tetap_jalan) {
    ssize_t len = read(inotify_fd, buf, sizeof(buf));
    if (len < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            usleep(200 * 1000);
            continue;
        } else if (errno == EINTR) {
            continue;
        } else {
            perror("read");
            break;
        }
    }
    if (len == 0) continue;

    ssize_t i = 0;
    while (i < len) {
        struct inotify_event *event = (struct inotify_event *)&buf[i];
        if (event->len) {
            time_t t = time(NULL);
            struct tm lt;
            localtime_r(&t, &lt);
            char waktu[64];
            strftime(waktu, sizeof(waktu), "%Y-%m-%d %H:%M:%S", &lt);

            const char *tipe = tipe_event(event->mask);
            const char *pidstr = "-";

            char baris_log[1024];
            int l = snprintf(baris_log, sizeof(baris_log), "[%s] [%s] [%s] [PID:%s]\n", waktu, tipe, event->name, pidstr);
            if (l > 0) {
                if (tambah_log(path_log, baris_log) != 0) {
                    dprintf(STDERR_FILENO, "Gagal menulis ke file log %s\n", path_log);
                }
            }
        }
    }
}

```

```

        dprintf(STDERR_FILENO, "Gagal menulis ke file log %s\n", path_log);
    }
}

char subjek[256];
char isi[512];
snprintf(subjek, sizeof(subjek), "File %s: %s", tipe, event->name);
snprintf(isi, sizeof(isi), "Peristiwa: %s\nFile: %s\nWaktu: %s\nLog tersimpan di: %s\n", tipe, event->name, waktu, path_log);
simulasi_email("dummy@example.com", subjek, isi);

dprintf(STDOUT_FILENO, "%s", baris_log);
}
i += sizeof(struct inotify_event) + event->len;
}

if (inotify_fd >= 0) close(inotify_fd);
dprintf(STDOUT_FILENO, "Program dihentikan. inotify fd ditutup.\n");
return 0;
}

```

- Dan ini adalah isi Log File beserta Output yang dihasilkan dari kode source diatas.

```
root@RIDHO:~# ./file_monitor
Memantau direktori: /tmp/monitor_dir
Mencatat ke: ./file_monitor.log
Hentikan dengan SIGTERM (kill -TERM <pid>)
```

```
root@RIDHO:~# ./file_monitor
Memantau direktori: /tmp/monitor_dir
Mencatat ke: ./file_monitor.log
Hentikan dengan SIGTERM (kill -TERM <pid>)
--- SIMULASI EMAIL DUMMY ---
Kepada: dummy@example.com
Subjek: File BUAT: testfile.txt
Isi Pesan:
Peristiwa: BUAT
File: testfile.txt
Waktu: 2025-10-14 22:40:39
Log tersimpan di: ./file_monitor.log

--- AKHIR SIMULASI EMAIL ---
[2025-10-14 22:40:39] [BUAT] [testfile.txt] [PID:-]
--- SIMULASI EMAIL DUMMY ---
Kepada: dummy@example.com
Subjek: File UBAH: testfile.txt
Isi Pesan:
Peristiwa: UBAH
File: testfile.txt
Waktu: 2025-10-14 22:41:15
Log tersimpan di: ./file_monitor.log

--- AKHIR SIMULASI EMAIL ---
[2025-10-14 22:41:15] [UBAH] [testfile.txt] [PID:-]
--- SIMULASI EMAIL DUMMY ---
Kepada: dummy@example.com
Subjek: File HAPUS: testfile.txt
Isi Pesan:
Peristiwa: HAPUS
File: testfile.txt
Waktu: 2025-10-14 22:41:29
Log tersimpan di: ./file_monitor.log

--- AKHIR SIMULASI EMAIL ---
[2025-10-14 22:41:29] [HAPUS] [testfile.txt] [PID:-]
```

```
root@RIDHO:~# touch /tmp/monitor_dir/testfile.txt
root@RIDHO:~# echo "hello" > /tmp/monitor_dir/testfile.txt
root@RIDHO:~# rm /tmp/monitor_dir/testfile.txt
root@RIDHO:~#
```

3. Pembahasan

- Program berhasil memantau perubahan file secara real-time menggunakan inotify.
- Event yang terdeteksi dicatat dengan format yang sesuai ke log file.
- Karena inotify tidak mendukung informasi PID, program mencatat PID:-.
- Untuk NIM ganjil, notifikasi dummy berhasil ditampilkan di layar terminal (simulasi email).
- Program berjalan hingga menerima sinyal SIGTERM, lalu keluar dengan aman.

KESIMPULAN

System call inotify dapat digunakan untuk memantau perubahan file atau direktori secara real time, Lalu program berhasil mencatat event ke file log menggunakan open() dan write(). Dan juga variasi NIM ganjil menambahkan fitur notifikasi dummy setiap event yang terjadi. Dan terakhir program dapat dihentikan dengan aman menggunakan SIGTERM dan menutup resource dengan benar.

**LAPORAN PRAKTIKUM
(TUGAS 3 SISTEM OPERASI)**



**Dosen Pengampu:
Triawan Adi Cahyanto M.Kom**

**Disusun oleh:
MUHAMMAD RIDHO (2410651009)**

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

2025

1. Tujuan Praktikum

- Memahami konsep dasar kerja shell dalam sistem operasi.
- Mempelajari cara menjalankan perintah (command) melalui proses `fork()` dan `execvp()`.
- Mengimplementasikan fitur piping, redirection, dan background process menggunakan system call.
- Menguji kemampuan manajemen proses dan I/O redirection pada program shell buatan sendiri.

2. Dasar Teori

Shell adalah program antarmuka antara pengguna dan kernel yang menerima perintah dari user, lalu mengeksekusinya. Beberapa konsep penting dalam implementasi shell antara lain:

- **fork()**: membuat proses anak (child process).
- **execvp()**: mengeksekusi program lain di dalam proses anak.
- **wait()** / **waitpid()**: menunggu proses anak selesai dieksekusi.
- **dup2()**: mengalihkan (redirect) input/output ke file atau pipe.
- **pipe()**: membuat saluran komunikasi antar proses.
- **chdir()**, **getcwd()**: digunakan untuk built-in command `cd` dan `pwd`.

Shell buatan sendiri harus mampu memproses input pengguna, memarsing perintah, dan menjalankan eksekusi dengan kontrol penuh terhadap proses dan file descriptor.

3. Ini adalah kode source dan output yang di inginkan beserta cara kompilasi yang berhasil, lalu eksekusi dan perintah yang benar dan yang terakhir perintah built-in seperti echo, pwd dan exit lalu menghasilkan valgrind tanpa error.

```
GNU nano 4.8 custom_shell.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <errno.h>

#define MAKS_BARIS 4096
#define MAKS_TOKEN 512
#define MAKS_PERINTAH 32

char *hapus_spasi(char *s) {
    while (*s == ' ' || *s == '\t' || *s == '\n') s++;
    if (*s == 0) return s;
    char *akhir = s + strlen(s) - 1;
    while (akhir > s && (*akhir == ' ' || *akhir == '\t' || *akhir == '\n')) akhir--;
    *(akhir+1) = 0;
    return s;
}

int tokenisasi(char *baris, char *token[]) {
    int jumlah = 0;
    char *p = baris;
    while (*p) {
        while (*p == ' ' || *p == '\t' || *p == '\n') p++;
        if (!*p) break;
        if (*p == '>' || *p == '<' || *p == '|' || *p == '&') {
            if (*p == '>' && *(p+1) == '>') {
                token[jumlah++] = ">>";
                p += 2;
            } else {
                char *t = (char*)malloc(3);
                t[0] = *p; t[1] = 0; t[2] = 0;
                token[jumlah++] = t;
                p++;
            }
        }
    }
}
```

```
GNU nano 4.8 custom_shell.c
    } else {
        char *mulai = p;
        while (*p && *p != ' ' && *p != '\t' && *p != '\n' && *p != '>' && *p != '<' && *p != '|' && *p != '&') p++;
        int panjang = p - mulai;
        char *t = (char*)malloc(panjang+1);
        strncpy(t, mulai, panjang);
        t[panjang] = 0;
        token[jumlah++] = t;
    }
    if (jumlah >= MAKS_TOKEN-1) break;
}
token[jumlah] = NULL;
return jumlah;
}

void bebas_token(char *token[], int jumlah) {
    for (int i=0; i<jumlah; i++) {
        if (strcmp(token[i], ">>") == 0) continue;
        free(token[i]);
    }
}

struct Perintah {
    char *argv[MAKS_TOKEN];
    char *file_masuk;
    char *file_keluar;
    int tambahkan;
};

int parsing_perintah(char *token[], int jumlah, struct Perintah cmd[]) {
    int i_cmd = 0;
    int i_arg = 0;
    for (int i=0; i<MAKS_PERINTAH; i++) { cmd[i].argv[0]=NULL; cmd[i].file_masuk=NULL; cmd[i].file_keluar=NULL; cmd[i].tambahkan=0; }
    for (int i=0; i<jumlah; i++) {
        char *t = token[i];
    }
}
```

```

GNU nano 4.8 custom_shell.c
    if (strcmp(t, "|") == 0) {
        cmd[i_cmd].argv[i_arg] = NULL;
        i_cmd++;
        i_arg = 0;
    } else if (strcmp(t, "<") == 0) {
        if (i+1 < jumlah) {
            cmd[i_cmd].file_masuk = strdup(token[++i]);
        }
    } else if (strcmp(t, ">") == 0) {
        if (i+1 < jumlah) {
            cmd[i_cmd].file_keluar = strdup(token[++i]);
            cmd[i_cmd].tambahkan = 0;
        }
    } else if (strcmp(t, ">>") == 0) {
        if (i+1 < jumlah) {
            cmd[i_cmd].file_keluar = strdup(token[++i]);
            cmd[i_cmd].tambahkan = 1;
        }
    } else if (strcmp(t, "&") == 0) {
        cmd[i_cmd].argv[i_arg++] = strdup("&");
    } else {
        cmd[i_cmd].argv[i_arg++] = strdup(t);
    }
}
cmd[i_cmd].argv[i_arg] = NULL;
return i_cmd+1;
}

int perintah_internal(char *cmd) {
    if (!cmd) return 0;
    return (strcmp(cmd, "cd") == 0 || strcmp(cmd, "exit") == 0 || strcmp(cmd, "pwd") == 0 || strcmp(cmd, "echo") == 0);
}

int jalankan_internal(char **argv) {
    if (strcmp(argv[0], "cd") == 0) {
        char *dir = argv[1];
        if (!dir) dir = getenv("HOME");
    }
}

```

```

GNU nano 4.8 custom_shell.c
    if (chdir(dir) != 0) perror("cd");
    return 1;
} else if (strcmp(argv[0], "pwd") == 0) {
    char cwd[4096];
    if (getcwd(cwd, sizeof(cwd))) printf("%s\n", cwd);
    else perror("pwd");
    return 1;
} else if (strcmp(argv[0], "exit") == 0) {
    int kode = 0;
    if (argv[1]) kode = atoi(argv[1]);
    exit(kode);
} else if (strcmp(argv[0], "echo") == 0) {
    for (int i=1; argv[i]; i++) {
        if (i>1) printf(" ");
        printf("%s", argv[i]);
    }
    printf("\n");
    return 1;
}
return 0;
}

void jalankan_pipeline(struct Perintah cmd[], int jumlah_cmd, int latar_belakang) {
    int i;
    int pipefds[2*(MAKS_PERINTAH)];
    for (i=0; i<jumlah_cmd-1; i++) {
        if (pipe(pipefds + i*2) < 0) {
            perror("pipe");
            return;
        }
    }

    pid_t pids[MAKS_PERINTAH];
    for (i=0; i<jumlah_cmd; i++) {
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork");
        }
    }
}

```

```

GNU nano 4.8 custom_shell.c
    return;
}
if (pid == 0) {
    if (i > 0) {
        dup2(pipefds[(i-1)*2], STDIN_FILENO);
    }
    if (i < jumlah_cmd-1) {
        dup2(pipefds[i*2 + 1], STDOUT_FILENO);
    }
    for (int j=0; j<2*(jumlah_cmd-1); j++) close(pipefds[j]);

    if (cmd[i].file_masuk) {
        int fd = open(cmd[i].file_masuk, O_RDONLY);
        if (fd < 0) { perror(cmd[i].file_masuk); exit(1); }
        dup2(fd, STDIN_FILENO);
        close(fd);
    }
    if (cmd[i].file_keluar) {
        int fd;
        if (cmd[i].tambahkan) fd = open(cmd[i].file_keluar, O_WRONLY|O_CREAT|O_APPEND, 0644);
        else fd = open(cmd[i].file_keluar, O_WRONLY|O_CREAT|O_TRUNC, 0644);
        if (fd < 0) { perror(cmd[i].file_keluar); exit(1); }
        dup2(fd, STDOUT_FILENO);
        close(fd);
    }

    if (jumlah_cmd==1 && perintah_internal(cmd[i].argv[0])) {
        jalankan_internal(cmd[i].argv);
        exit(0);
    }

    int argc = 0;
    while (cmd[i].argv[argc] argc++;
    if (argc>0 && strcmp(cmd[i].argv[argc-1], "&") == 0) { free(cmd[i].argv[argc-1]); cmd[i].argv[argc-1]=NULL; }

    if (cmd[i].argv[0]==NULL) exit(0);
    execvp(cmd[i].argv[0], cmd[i].argv);
}

```

```

GNU nano 4.8 custom_shell.c
        fprintf(stderr, "%s: perintah tidak ditemukan\n", cmd[i].argv[0]);
        exit(127);
    } else {
        pids[i] = pid;
    }
}

for (int j=0; j<2*(jumlah_cmd-1); j++) close(pipefds[j]);

if (!latar_belakang) {
    for (i=0; i<jumlah_cmd; i++) {
        int status;
        waitpid(pids[i], &status, 0);
    }
} else {
    for (i=0; i<jumlah_cmd; i++) printf("[latar belakang] pid %d\n", pids[i]);
}
}

int main() {
    char baris[MAKS_BARIS];
    while (1) {
        printf("shellku> ");
        fflush(stdout);
        if (!fgets(baris, sizeof(baris), stdin)) { printf("\n"); break; }
        char *input = hapus_spasi(baris);
        if (strlen(input)==0) continue;

        char *token[MAKS_TOKEN];
        int jumlah = tokenisasi(input, token);
        if (jumlah==0) continue;

        int latar_belakang = 0;
        if (jumlah>0 && strcmp(token[jumlah-1], "&") == 0) {
            latar_belakang = 1;
            free(token[jumlah-1]); token[jumlah-1] = NULL; jumlah--;
        }
    }
}

```

```

struct Perintah cmd[MAKS_PERINTAH];
int jumlah_cmd = parsing_perintah(token, jumlah, cmd);
if (jumlah_cmd <= 0) { bebas_token(token, jumlah); continue; }

int tunggal = (jumlah_cmd==1);
if (tunggal && cmd[0].file_masuk==NULL && cmd[0].file_keluar==NULL && perintah_internal(cmd[0].argv[0])) {
    int argc=0; while (cmd[0].argv[argc]) argc++;
    if (argc>0 && strcmp(cmd[0].argv[argc-1], "&")==0) { free(cmd[0].argv[argc-1]); cmd[0].argv[argc-1]=NULL; }
    jalankan_internal(cmd[0].argv);
} else {
    jalankan_pipeline(cmd, jumlah_cmd, latar_belakang);
}

for (int i=0; i<jumlah_cmd; i++) {
    for (int j=0; cmd[i].argv[j]; j++) free(cmd[i].argv[j]);
    if (cmd[i].file_masuk) free(cmd[i].file_masuk);
    if (cmd[i].file_keluar) free(cmd[i].file_keluar);
}
bebas_token(token, jumlah);
return 0;
}

```

- Ini adalah output dari kode source diatas.

```

root@RIDHO:~# nano custom_shell.c
root@RIDHO:~# gcc -o custom_shell custom_shell.c
root@RIDHO:~# ./custom_shell
shellku> ls | grep txt > output.txt &
[latar belakang] pid 59
[latar belakang] pid 60
shellku> valgrind --leak-check=full ./myshell
valgrind: ./myshell: No such file or directory
shellku> valgrind --leak-check=full ./custom_shell
==62== Memcheck, a memory error detector
==62== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==62== Using Valgrind-3.15.0 and LibVEX; rerun with -h for copyright info
==62== Command: ./custom_shell
==62==
shellku> ls | grep txt > hasil.txt &
[latar belakang] pid 63
[latar belakang] pid 64
shellku> echo Halo Dunia
Halo Dunia
shellku> pwd
/root
shellku> exit
==62==
==62== HEAP SUMMARY:
==62==   in use at exit: 10 bytes in 2 blocks
==62==   total heap usage: 23 allocs, 21 frees, 2,151 bytes allocated
==62==
==62== LEAK SUMMARY:
==62==   definitely lost: 0 bytes in 0 blocks
==62==   indirectly lost: 0 bytes in 0 blocks
==62==   possibly lost: 0 bytes in 0 blocks
==62==   still reachable: 10 bytes in 2 blocks
==62==   suppressed: 0 bytes in 0 blocks
==62== Reachable blocks (those to which a pointer was found) are not shown.
==62== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==62==
==62== For lists of detected and suppressed errors, rerun with: -s
==62== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
shellku> |

```

4. Pembahasan

- Program berhasil membaca input dan memisahkan token dengan strtok().
- Implementasi fork() dan execvp() bekerja sesuai ekspektasi.
- Piping dan redirection berjalan lancar tanpa error.
- Valgrind menunjukkan tidak ada kkebocoran memori.
- Program memenuhi semua kriteria tugas, termasuk multiple pipes untuk NIM ganjil.

Kesimpulan

Dari percobaan ini dapat disimpulkan bahwa:

- Mahasiswa memahami konsep dasar shell internal, process control, dan I/O redirection.
- Shell buatan sendiri dapat menjalankan perintah layaknya terminal asli.
- Tidak terdapat error maupun memory leak selama eksekusi program.
- Tujuan praktikum telah tercapai sepenuhnya.

**LAPORAN PRAKTIKUM
(TUGAS 4 SISTEM OPERASI)**



Dosen Pengampu:
Adi Triawan Cahyanto M.Kom

Disusun oleh:
MUHAMMAD RIDHO (2410651009) (A)

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

2025

1. Tujuan Praktikum

- Memahami struktur dan fungsi direktori **/proc** pada sistem operasi Linux.
- Menggunakan **system calls** seperti `open()`, `read()`, dan `statvfs()` untuk membaca informasi sistem secara langsung.
- Menampilkan informasi CPU, memori, disk, proses, dan jaringan tanpa menggunakan library tambahan seperti `sysinfo`.
- Mengembangkan kemampuan parsing data dan format output tabel ASCII.
- (Variasi unik) Menampilkan informasi **network socket** dari `/proc/net/tcp` berdasarkan digit ganjil pada NIM.

2. Dasar Teori

- **/proc Filesystem**

`/proc` adalah virtual filesystem yang digunakan kernel Linux untuk menyediakan informasi sistem secara dinamis.

File seperti `/proc/cpuinfo`, `/proc/meminfo`, dan `/proc/net/dev` menyimpan data tentang CPU, memori, dan jaringan.

- **System Call**

Adalah antarmuka antara program pengguna dan kernel. Contohnya:

- a. `open()`, `read()`, `close()` → membaca file sistem
- b. `statvfs()` → mendapatkan informasi ruang disk

- **Parsing File /proc**

Data di `/proc` berupa teks, sehingga program harus melakukan parsing manual menggunakan `sscanf()` atau `strtok()`.

- **Tabel ASCII Output**

Digunakan agar hasil terlihat terstruktur tanpa library tambahan.

- Ini adalah kode source nya

```

GNU nano 4.8                                     sistem_informasi.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/statvfs.h>
#include <sys/types.h>
#include <ctype.h>

void baca_cpuinfo() {
    int fd = open("/proc/cpuinfo", O_RDONLY);
    if (fd < 0) return;
    char buffer[8192];
    int bytes = read(fd, buffer, sizeof(buffer) - 1);
    buffer[bytes] = '\0';
    close(fd);

    char *baris = strtok(buffer, "\n");
    char model[256] = "";
    char core[64] = "";
    char freq[64] = "";
    while (baris) {
        if (strncmp(baris, "model name", 11) == 0) strcpy(model, strchr(baris, ':') + 2);
        else if (strncmp(baris, "cpu cores", 9) == 0) strcpy(core, strchr(baris, ':') + 2);
        else if (strncmp(baris, "cpu MHz", 7) == 0) strcpy(freq, strchr(baris, ':') + 2);
        baris = strtok(NULL, "\n");
    }
    printf("+----- CPU INFO -----+\n");
    printf("| Model CPU       : %-45s |\n", model);
    printf("| Jumlah Core    : %-45s |\n", core);
    printf("| Frekuensi (MHz) : %-45s |\n", freq);
    printf("+-----+\n\n");
}

```

```

GNU nano 4.8                                     sistem_informasi.c
void baca_meminfo() {
    int fd = open("/proc/meminfo", O_RDONLY);
    if (fd < 0) return;
    char buffer[4096];
    int bytes = read(fd, buffer, sizeof(buffer) - 1);
    buffer[bytes] = '\0';
    close(fd);

    long total = 0, bebas = 0;
    char *baris = strtok(buffer, "\n");
    while (baris) {
        if (strncmp(baris, "MemTotal", 8) == 0) sscanf(baris, "MemTotal: %ld", &total);
        else if (strncmp(baris, "MemAvailable", 12) == 0) sscanf(baris, "MemAvailable: %ld", &bebas);
        baris = strtok(NULL, "\n");
    }
    long terpakai = total - bebas;
    printf("+----- MEMORI INFO -----+\n");
    printf("| Total Memori   : %-12ld KB |\n", total);
    printf("| Memori Bebas   : %-12ld KB |\n", bebas);
    printf("| Memori Terpakai : %-12ld KB |\n", terpakai);
    printf("+-----+\n\n");
}

void baca_diskinfo() {
    struct statvfs data;
    if (statvfs("/", &data) != 0) return;
    unsigned long total = data.f_blocks * data.f_frsize / 1024;
    unsigned long bebas = data.f_bfree * data.f_frsize / 1024;
    unsigned long terpakai = total - bebas;
    printf("+----- DISK INFO -----+\n");
    printf("| Total Disk    : %-12lu KB |\n", total);
    printf("| Disk Bebas    : %-12lu KB |\n", bebas);
    printf("| Disk Terpakai : %-12lu KB |\n", terpakai);
    printf("+-----+\n\n");
}

```

```

GNU nano 4.8                                     sistem_informasi.c
void baca_proses() {
    DIR *direktori = opendir("/proc");
    if (!direktori) return;
    struct dirent *entri;
    printf("+----- PROSES AKTIF -----+\n");
    printf("| PID | Nama Proses | Status | \n");
    printf("+-----+\n");
    while ((entri = readdir(direktori)) != NULL) {
        if (!isdigit(entri->d_name[0])) continue;
        char jalur[512];
        snprintf(jalur, sizeof(jalur), "/proc/%s/status", entri->d_name);
        int fd = open(jalur, O_RDONLY);
        if (fd < 0) continue;
        char buffer[1024];
        int bytes = read(fd, buffer, sizeof(buffer) - 1);
        buffer[bytes] = '\0';
        close(fd);

        char nama[128] = "", status[64] = "";
        char *baris = strtok(buffer, "\n");
        while (baris) {
            if (strncmp(baris, "Name:", 5) == 0) strcpy(nama, strchr(baris, '\t') + 1);
            else if (strncmp(baris, "State:", 6) == 0) strcpy(status, strchr(baris, '\t') + 1);
            baris = strtok(NULL, "\n");
        }
        printf("| %-7s | %-28s | %-20s | \n", entri->d_name, nama, status);
    }
    closedir(direktori);
    printf("+-----+\n\n");
}

```

```

GNU nano 4.8                                     sistem_informasi.c
void baca_tcpinfo() {
    int fd = open("/proc/net/tcp", O_RDONLY);
    if (fd < 0) return;
    char buffer[8192];
    int bytes = read(fd, buffer, sizeof(buffer) - 1);
    buffer[bytes] = '\0';
    close(fd);

    printf("+----- SOCKET TCP -----+\n");
    printf("| Local Address | Remote Address | Status | \n");
    printf("+-----+\n");

    char *baris = strtok(buffer, "\n");
    int hitung = 0;
    while (baris) {
        if (hitung++ == 0) { baris = strtok(NULL, "\n"); continue; }
        char local[64], remote[64], state[8];
        sscanf(baris, "%*d: %64s %64s %2s", local, remote, state);
        printf("| %-18s | %-18s | %-17s | \n", local, remote, state);
        baris = strtok(NULL, "\n");
    }
    printf("+-----+\n\n");
}

```

```

GNU nano 4.8                                     sistem_informasi.c
void baca_jaringan() {
    int fd = open("/proc/net/dev", O_RDONLY);
    if (fd < 0) return;
    char buffer[4096];
    int bytes = read(fd, buffer, sizeof(buffer) - 1);
    buffer[bytes] = '\0';
    close(fd);

    printf("+----- ANTARMUKA JARINGAN -----+\n");
    printf("| Nama Interface | Bytes Diterima | Bytes Dikirim |+\n");
    printf("+-----+\n");

    char *baris = strtok(buffer, "\n");
    int hitung = 0;
    while (baris) {
        if (hitung++ < 2) { baris = strtok(NULL, "\n"); continue; }
        char nama[32];
        unsigned long terima, kirim;
        sscanf(baris, "%[^:]: %lu %u %u %u %u %u %u %u", nama, &terima, &kirim);
        printf("| %-14s | %-14lu | %-25lu |+\n", nama, terima, kirim);
        baris = strtok(NULL, "\n");
    }
    printf("+-----+\n\n");
}

```

```

int main(int argc, char *argv[]) {
    int ganjil = 0;
    if (argc > 2 && strcmp(argv[1], "-n") == 0) {
        int nim = atoi(argv[2]);
        ganjil = nim % 2;
    }
    baca_cpuinfo();
    baca_meminfo();
    baca_diskinfo();
    baca_proses();
    baca_jaringan();
    if (ganjil) baca_tcpinfo();
    return 0;
}

```

- Dan ini adalah output yang dihasilkan dari kode source diatas

```
root@RIDHO:~# nano sistem_informasi.c
root@RIDHO:~# gcc -o sistem_informasi sistem_informasi.c
root@RIDHO:~# ./sistem_informasi
+----- CPU INFO -----+
| Model CPU      :          |
| Jumlah Core    : 10       |
| Frekuensi (MHz) : 2688.010 |
+-----+
+----- MEMORI INFO -----+
| Total Memori : 7951280      KB |
| Memori Bebas : 7633044      KB |
| Memori Terpakai : 318236     KB |
+-----+
+----- DISK INFO -----+
| Total Disk : 263174212      KB |
| Disk Bebas : 261233168      KB |
| Disk Terpakai : 1941044      KB |
+-----+
+----- PROSES AKTIF -----+
| PID | Nama Proses | Status |
+-----+
| 1    | init        | S (sleeping) |
| 72   | init        | S (sleeping) |
| 73   | init        | S (sleeping) |
| 74   | bash        | S (sleeping) |
| 115  | sistem_informas | R (running)  |
+-----+
+----- ANTARMUKA JARINGAN -----+
| Nama Interface | Bytes Diterima | Bytes Dikirim |
+-----+
| lo              | 0              | 0              |
| bond0           | 0              | 0              |
| dummy0          | 0              | 0              |
| tunl0           | 0              | 0              |
| sit0            | 0              | 0              |
| eth0            | 50261          | 1216           |
+-----+
```

3. Hasil dan Pembahasan

1. Hasil Output Program

- Tampilkan screenshot hasil eksekusi program (./sistem_informasi).
- Jelaskan bahwa program berhasil menampilkan informasi seperti:
 - Model CPU, jumlah core, dan frekuensi
 - Total dan penggunaan memori
 - Total, used, free disk
 - Daftar proses yang sedang berjalan
 - Interface jaringan dan traffic (RX/TX)
 - (Variasi unik) Socket aktif di /proc/net/tcp

2. Analisis

- Program berjalan sesuai fungsi masing-masing.
- Peringatan saat kompilasi telah diperbaiki dengan menambahkan `#include <ctype.h>` dan memperbesar buffer.
- Semua informasi sesuai dengan hasil dari perintah sistem Linux.

KESIMPULAN

1. Direktori /proc menyediakan berbagai informasi sistem secara real-time.
2. Dengan menggunakan **system call** dasar (open, read, statvfs), mahasiswa dapat membuat tool monitoring sederhana tanpa library tambahan.
3. Program dapat menampilkan informasi CPU, memori, disk, proses, jaringan, dan socket dengan format tabel ASCII.
4. Praktikum ini memperdalam pemahaman hubungan antara **user space** dan **kernel space** dalam sistem operasi Linux.